

Raspberry Pi 5 - ESP32 기반 엣지 제어

시스템 구현 및 성능 분석

항목	내용
학교	아주대학교
학과	소프트웨어학과
이름	오재영
진행 기간	2026.07 ~ 2026.09
저장소	GitHub: edge-iot-control-system

1. 프로젝트를 시작한 이유.....	3
2. 구현한 시스템.....	4
2.1 Light Switch Node.....	5
2.2 PC Power Node.....	6
3. 실제 구현에서 마주친 문제.....	7
4. 성능 측정.....	8
5. 가장 기억에 남았던 결과: Wi-Fi Sleep.....	9
6. HTTP / MQTT 응답시간 비교.....	10
7. ESP32 내부 처리 속도.....	12
8. 의도된 Delay.....	13
9. 추가로 확인한 내용.....	14
9.1 ESP32 Heap.....	14
9.2 Raspberry Pi 자원 사용량.....	15
10. 마무리.....	16

1. 프로젝트를 시작한 이유

고등학생 때 Arduino Uno를 이용해 스마트 병실과 스마트 팜을 만들어본 경험이 있습니다. 작은 장치에 직접 코드를 작성하여 제가 원하는 기능을 동작하게 만드는 과정이 흥미롭게 느껴졌습니다.

2025년 겨울방학에는 실전코딩2를 수강하였습니다. 마지막 강의 시간에 조별 활동으로 ESP32를 활용한 무선 키보드를 구현해보면서, ESP32를 이용해 실제 생활에서 사용할 수 있는 장치를 직접 만들어보고 싶다는 생각이 들었습니다. 이를 계기로 이번 프로젝트를 시작하게 되었습니다.

제가 생각한 제어 장치는 벽 전등 스위치와 PC 전원 버튼이었습니다. 기존 배선이나 장치 내부 회로를 수정하기보다, 서보 모터가 사람 손처럼 물리적으로 스위치와 버튼을 누르는 방식을 선택했습니다.

처음에는 “실제로 장치가 작동하게 하는 것”을 목표로 두고 시작하였습니다.

하지만 기능을 하나씩 구현하고 실제 환경에서 반복하여 사용하면서, 단순히 Code가 정상적으로 실행되는 것과 실제 시스템이 안정적으로 동작하는 것은 서로 다른 문제라는 것을 알게 되었습니다.

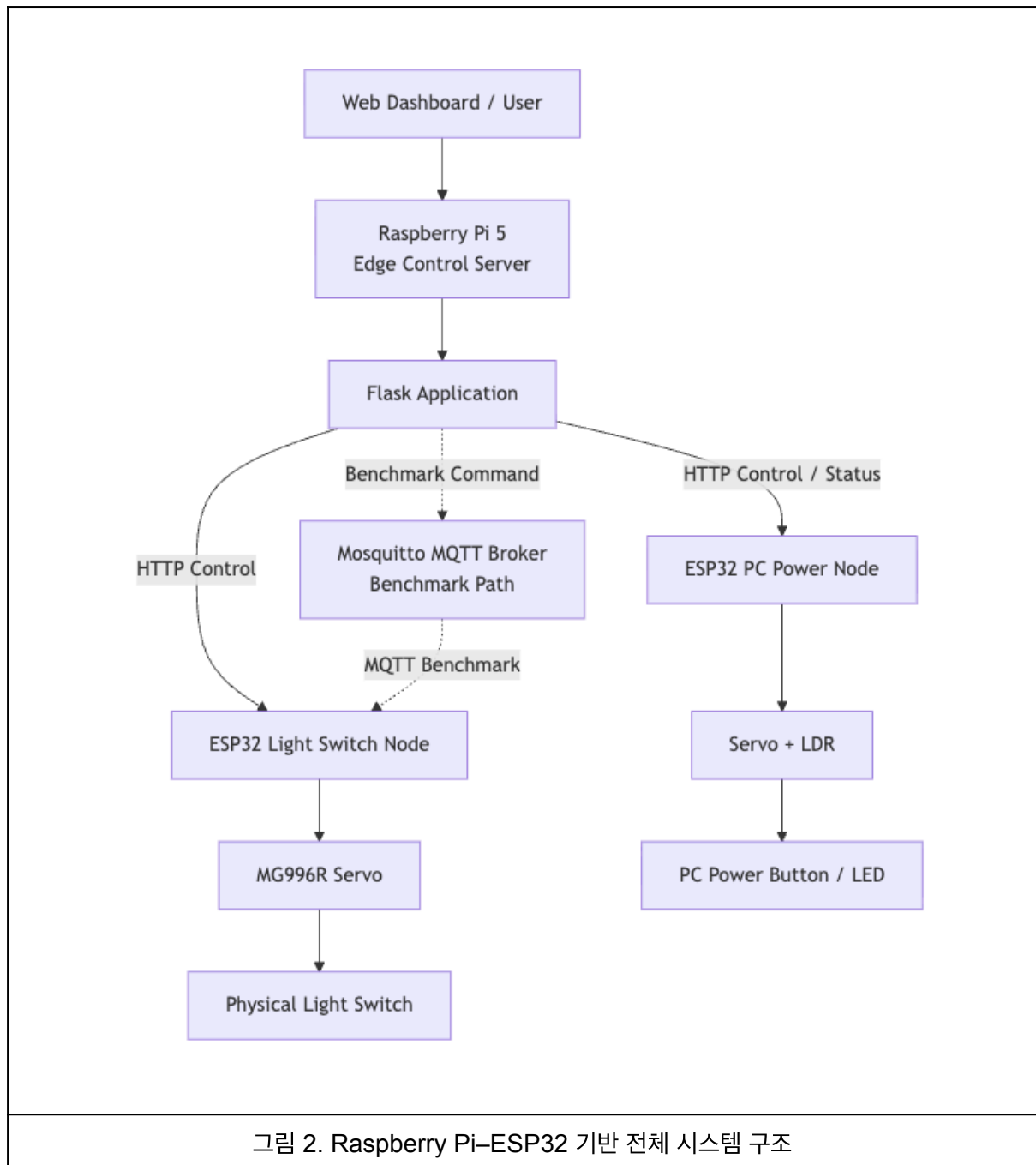
프로젝트를 진행하며 실패도 겪고, 생각과 다르게 작동하기도 하고, 더 나은 방법이 없을까 고민도 해보며 제어 장치를 완성했지만, “왜 실패하는지”, “시간이 어디에서 사용되는지”, “설정을 바꾸면 왜 결과가 달라지는지” 등 제가 겪은 경험을 분석하고 원인을 찾아보고 싶어서 단순히 시스템 구현이 아닌 구현 후 문제를 발견하고 측정과 분석으로 확장하게 되었습니다.



그림 1. 프로젝트 준비물

2. 구현한 시스템

전체 구조는 다음과 같습니다.



Raspberry Pi 5는 중앙 제어 서버 역할을 하고, ESP32는 실제 서보 모터와 센서가 연결된 제어 노드로 동작합니다.

실제 Light Switch와 PC Power 제어는 HTTP를 사용했고, MQTT는 이후 통신 방식에 따른 응답시간을 비교하기 위해 Benchmark 경로로 추가했습니다.

2.1 Light Switch Node

Light Switch Node는 MG996R 서보 모터가 실제 벽면 스위치를 눌렀다가 원래 위치로 돌아오는 구조입니다. 기존 스위치의 배선이나 회로를 변경하지 않고 서보 모터가 사람의 손처럼 Switch를 눌렀다가 원래 위치로 돌아오게 만들었습니다.

최종 설정은 다음과 같습니다.

구분	설정
REST Angle	90°
On Angle	50°
OFF Angle	140°
Press Hold	400 ms
Return Wait	600 ms

각도는 실제 스위치를 반복해서 눌러보며 조정하였습니다.

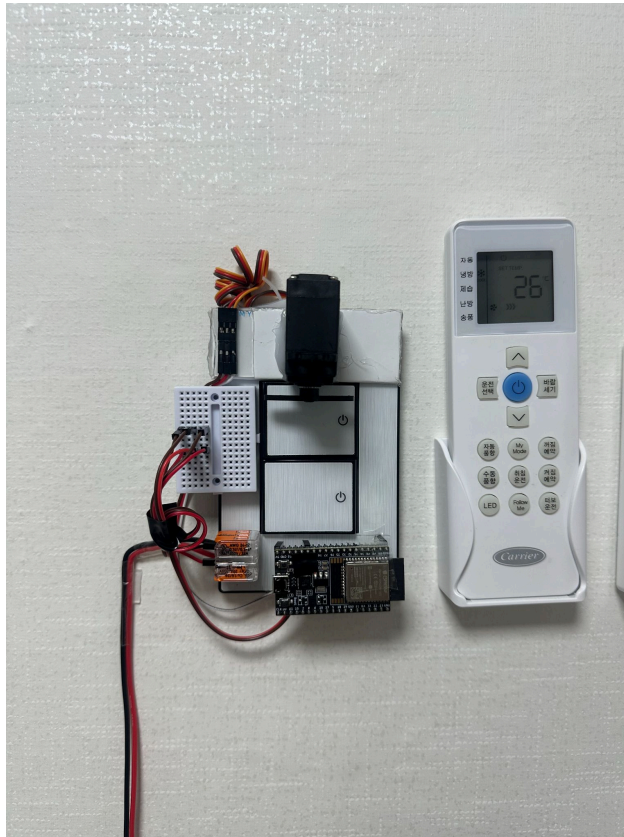


그림 3. 실제 벽면 전등 스위치를 제어하는 Light Switch Node

2.2 PC Power Node

PC 전원 버튼을 서보 모터로 누르는 기능을 구현하였습니다. 하지만 PC Power Button은 Light Switch와 달리 단순히 명령이 들어올 때마다 누르면 문제가 발생한다고 판단하였습니다. 이미 PC가 켜져 있는데 다시 Power Button을 누르면 사용자가 원하지 않는 종료 또는 절전 동작이 발생할 수 있기 때문입니다.

그래서 두 가지 정보를 이용해 PC 상태를 확인했습니다.

- Raspberry Pi에서 PC로 보내는 Ping
- ESP32에 연결한 LDR이 읽는 PC 전원 LED 밝기

최종 판단은 다음과 같이 하였습니다.

Ping 결과	LDR 평균값	LED 판단	PC 상태 판단
성공	< 3500	LED ON	PC ON
성공	≥ 3500	LED OFF	PC ON
실패	< 3500	LED ON	PC ON
실패	≥ 3500	LED OFF	PC OFF 후보

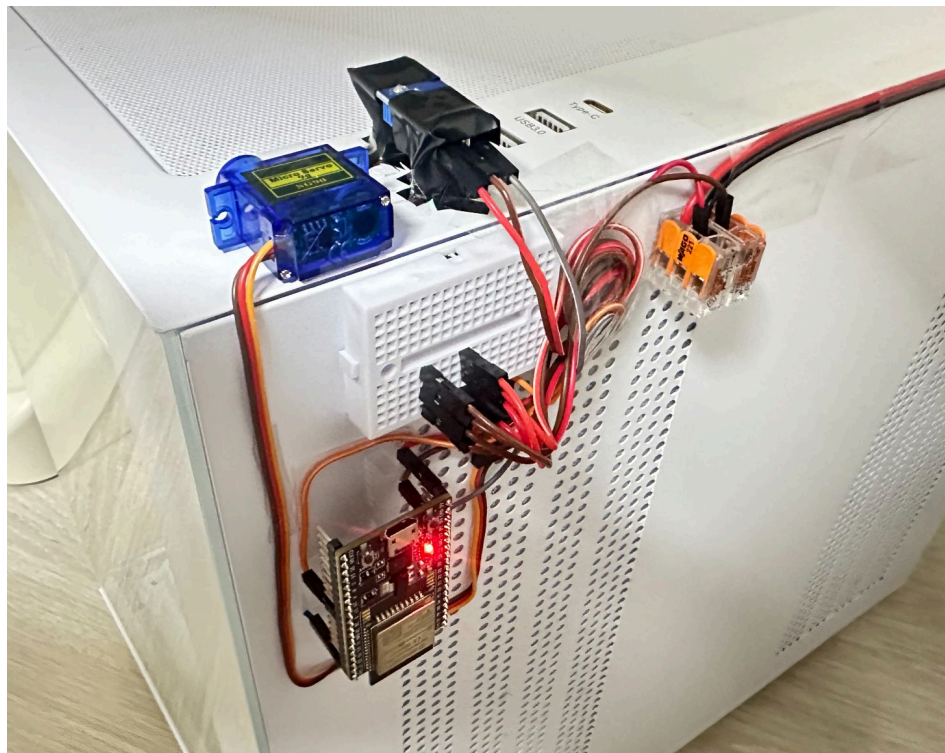


그림 4. Ping과 LDR를 이용한 상태 확인 기반 PC Power Node

3. 실제 구현에서 마주친 문제

초기 Light Switch에는 MG90S 서보 모터를 사용했습니다. Raspberry Pi에서 ESP32로 요청은 정상적으로 전달됐고, ESP32도 서보 모터에 명령을 실행했습니다. 하지만 실제 벽 스위치를 시험했을 때 결과는 달랐습니다.

MG90S			
결과	성공 횟수	시행 횟수	성공률
ON	20	20	100%
OFF	7	20	35%

소프트웨어에서는 정상적으로 신호를 보냈지만, 실제 스위치가 충분히 눌리지 않는 경우가 반복됐습니다.

처음에는 토크가 약해서 실패할 가능성이 있다고 생각했습니다. 그래서 기존 서보 MG90S에서 토크가 더 강한 MG996R로 교체했습니다. 이 상태에서 당시 확인한 동작에서는 잘 작동하는 것을 확인했습니다.

하지만 이후 힘이 커지면서 스위치를 필요 이상으로 깊게 누르는 경우가 있었고, 각도를 줄이면 반대로 충분히 눌리지 않는 경우가 있었습니다. 또한 반복해서 사용할 수 있도록 서보 자체가 움직이지 않게 고정을 더 안정적으로 만들 필요도 있었습니다.

그래서 이후에는 고정을 보강하고, 스위치에 정확히 힘이 전달되는 범위로 각도를 조정했습니다.

MG996R			
결과	성공 횟수	시행 횟수	성공률
ON	20	20	100%
OFF	20	20	100%

이 경험에서 가장 크게 느낀 점은 명령이 전달된 것과 실제 장치가 원하는 상태로 바뀐 것은 서로 다른 성공 조건이라는 점이었습니다.

처음에는 소프트웨어가 정상적으로 응답하면 제어도 성공했다고 생각하기 쉬웠는데, 실제 시스템에서는 하드웨어의 힘, 고정 상태, 접촉 위치와 같은 요소도 전체 결과를 결정했습니다.

4. 성능 측정

기능이 어느 정도 동작한 뒤에는 Raspberry Pi와 ESP32 사이에서 실제로 시간이 얼마나 걸리는지 궁금해졌습니다. 하지만 Light Switch 기능을 그대로 측정하면 의도적으로 넣은 서보 모터가 움직이는 시간까지 포함되기 때문에, 통신과 애플리케이션 응답 구간을 따로 보기 위한 Benchmark Endpoint를 만들었습니다.

이 경로에서는 다음 요소를 제외했습니다.

- 서보 모터 구동
- LDR 측정
- Ping
- Servo Hold / Return Delay

측정한 값은 Raspberry Pi가 요청을 시작한 시점부터 ESP32의 응답을 받을 때까지의 Application RTT입니다.

최종 실험은 다음과 같이 구성했습니다.

구분	시행 횟수
Warm-up	50회
Main Measurement	200회 x 5 Runs
Samples / Protocol	1000
Total Samples	3000
Wi-Fi Sleep	OFF

특정 방식이 항상 먼저 또는 마지막에 측정되는 것을 피하기 위해 Run마다 실행 순서를 바꾸었습니다.

최종 3000개의 요청은 모두 정상적으로 완료됐습니다.

5. 가장 기억에 남았던 결과: Wi-Fi Sleep

MQTT QoS 0를 처음 구현하고 측정했을 때 약 100 ms 수준의 RTT가 반복해서 나타났습니다. MQTT가 HTTP보다 훨씬 느린 것처럼 보였습니다. 예상했던 결과와 차이가 컸고 바로 “MQTT가 느리다”라고 결론을 내리기에는 결과가 이상하다고 느껴 다른 조건을 확인했습니다.

그 과정에서 ESP32의 Wi-Fi Sleep 설정이 Latency에 영향을 줄 가능성이 있다는 것을 알게 되었고, 이를 확인하기 위해 Wi-Fi Sleep 설정을 변수로 두고 ON/OFF 조건을 나누어 측정했습니다.

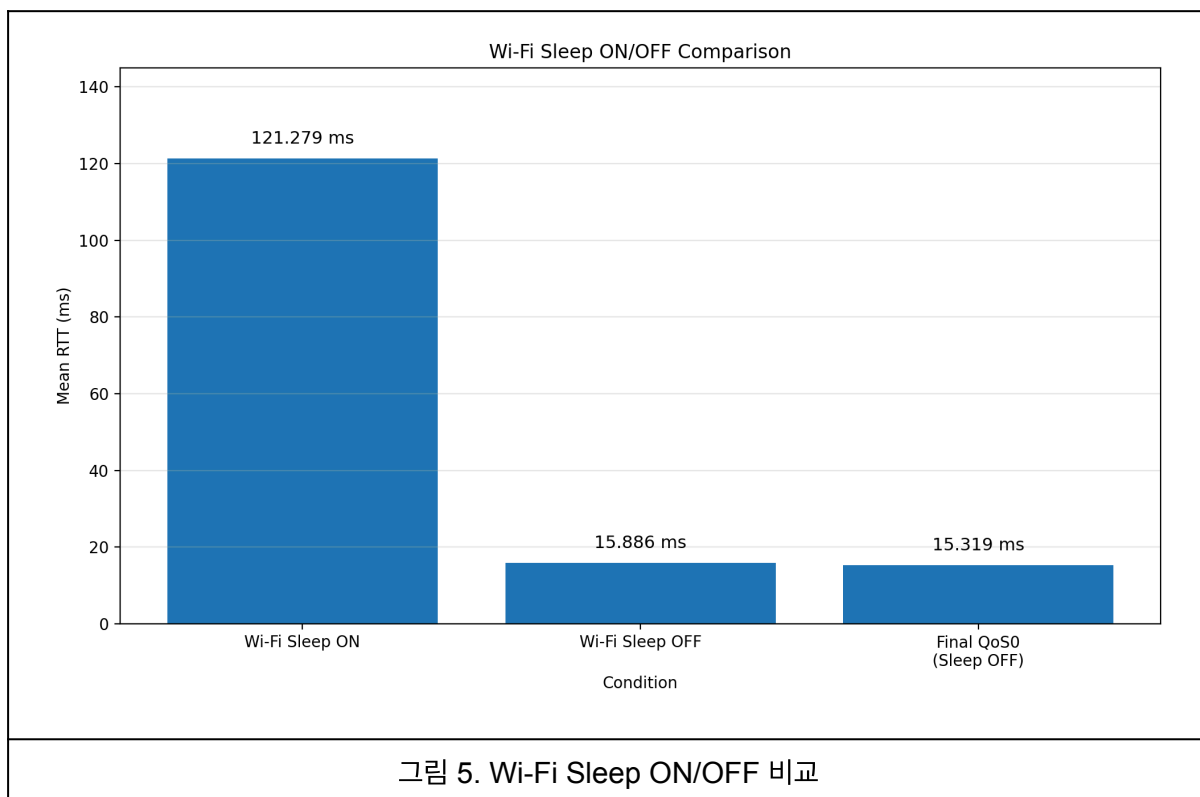
조건	평균 RTT
Wi-Fi Sleep ON	121.279 ms
Wi-Fi Sleep OFF	15.886 ms
최종 QoS 0 본실험	15.319 ms

WiFi.setSleep(false)를 적용한 뒤 평균 RTT가 약 121 ms에서 약 16 ms로 크게 감소했습니다.

이후 1000회 본실험에서도 평균 15.319 ms로 비슷한 수준이 다시 나타났습니다.

처음 나온 결과를 그대로 받아들이지 않고 조건을 바꾸어 다시 확인했더니 결과가 완전히 달라졌다는 것을 직접 경험할 수 있어서 개인적으로 이번 프로젝트에서 가장 흥미로운 기억으로 남아있습니다.

이후 최종 Protocol Benchmark는 모두 Wi-Fi Sleep OFF 조건으로 통일했습니다.



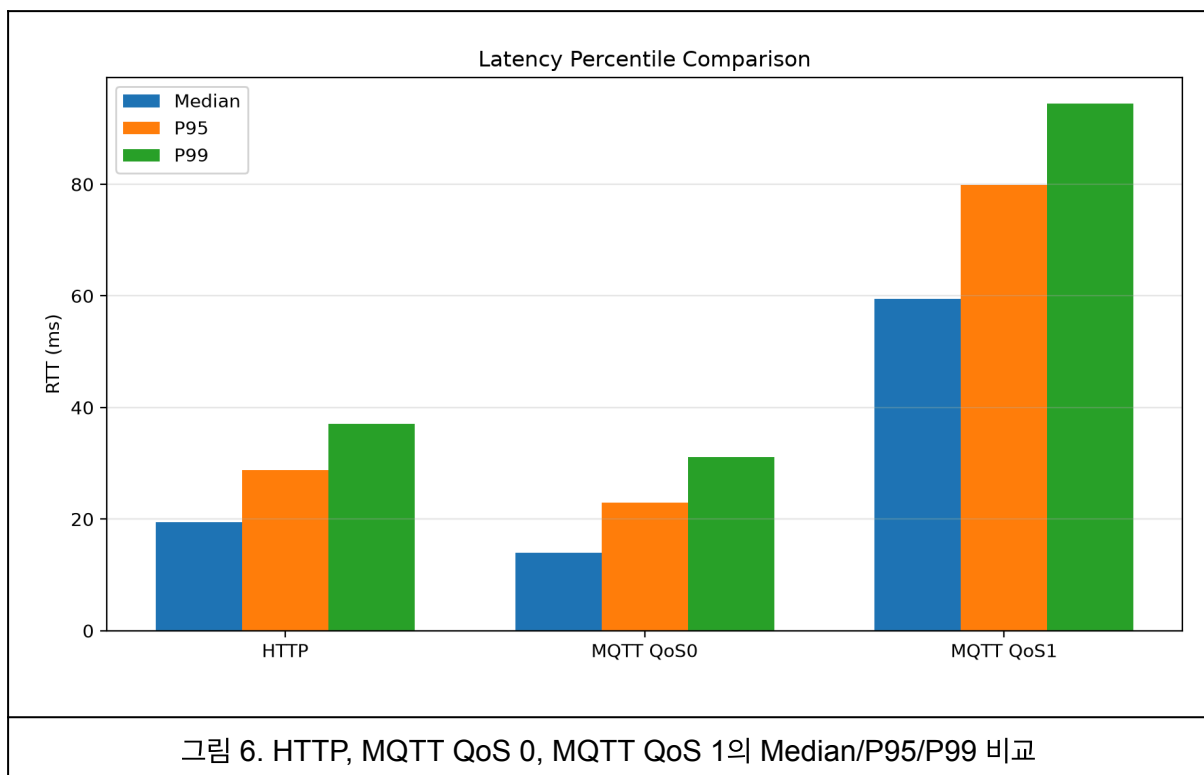
6. HTTP / MQTT 응답시간 비교

최종 결과는 다음과 같습니다.

Protocol	Mean	Median	P95	P99
HTTP	20.515 ms	19.450 ms	28.807 ms	37.038 ms
MQTT QoS 0	15.319 ms	13.988 ms	22.980 ms	31.076 ms
MQTT QoS 1	61.282 ms	59.508 ms	79.849 ms	94.444 ms

현재 시스템에서는 MQTT QoS 0가 가장 낮은 RTT를 보였고, MQTT QoS 1은 가장 높은 값을 보였습니다.

다만 HTTP와 MQTT의 구현 방식에는 차이가 있습니다. 현재 HTTP는 요청이 끝나면 연결을 종료하지만, MQTT는 연결을 유지한 상태에서 메시지를 주고받습니다. 따라서 해석 범위를 현재 시스템으로 한정하였습니다.

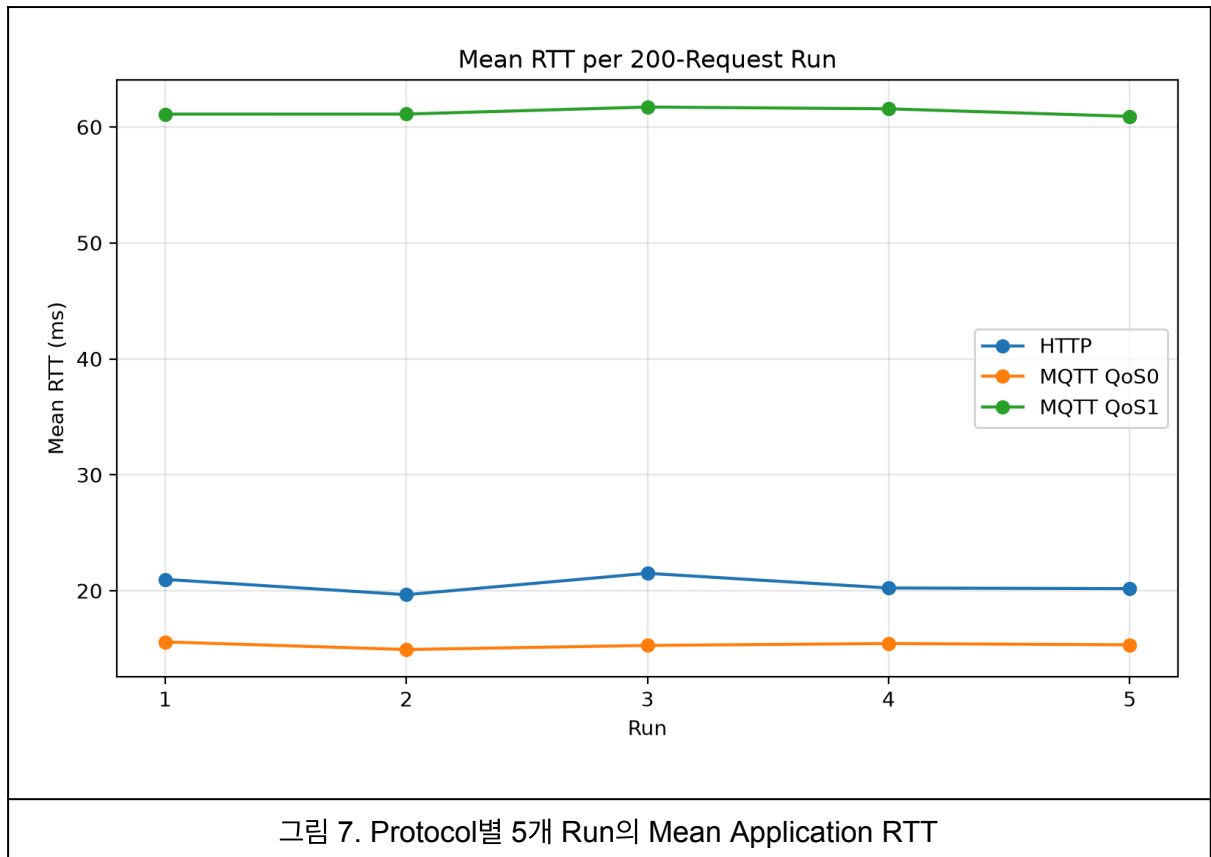


5개의 Run을 각각 확인한 결과도 비슷한 수준이 유지되었습니다.

HTTP의 Run Mean은 약 19.7~21.5 ms,

MQTT QoS 0는 약 14.9~15.6 ms,

MQTT QoS 1은 약 60.9~61.7 ms 범위였습니다.



7. ESP32 내부 처리 속도

HTTP와 MQTT의 RTT가 다르게 나타난 뒤에는 ESP32 내부에서 실제 계산에 얼마나 시간이 쓰이는지도 확인해봤습니다.

ESP32 Benchmark Handler 내부에 `micros()` 기반 타이머를 넣어 처리시간을 측정했습니다.

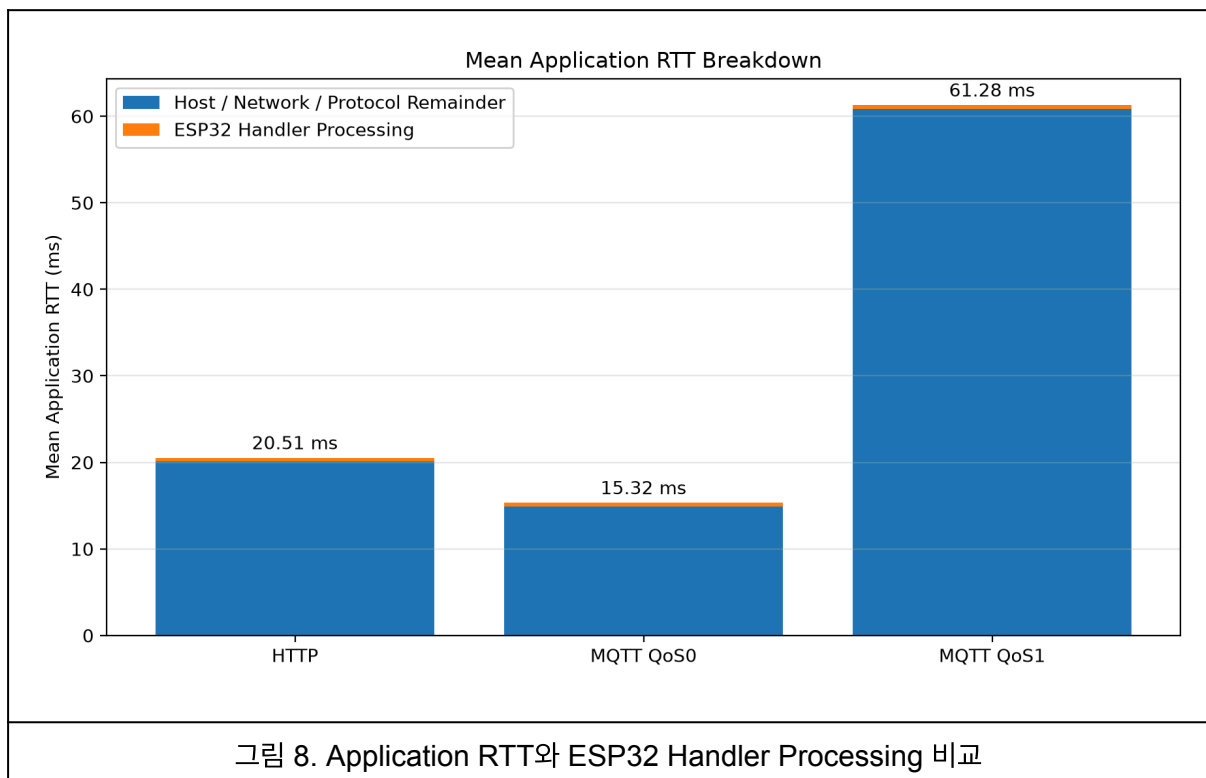
Protocol	ESP32 Handler 평균 처리시간
HTTP	401.999 μ s
MQTT QoS 0	419.632 μ s
MQTT QoS 1	420.765 μ s

세 조건 모두 약 0.4 ms로 비슷했습니다.

반면 전체 Application RTT는 약 15~61 ms였습니다. 그래서 적어도 이번에 측정한 ESP32 Handler 내부 계산시간만으로는 Protocol별 RTT 차이를 설명하기 어렵다고 판단했습니다. 나머지 시간에는 Raspberry Pi 측 처리, Wi-Fi 통신, TCP / MQTT 처리, Broker, ESP32 네트워크 처리 등 여러 부분이 함께 포함되어 있을 수 있습니다.

이번 프로젝트에서는 그 구간을 각각 따로 나누어 측정하지는 못했습니다.

특히 MQTT QoS 1이 QoS 0보다 평균 약 46 ms 높은 이유는 아직 확인하지 못했습니다.



8. 의도된 Delay

Protocol Benchmark에서는 약 15 ~ 61 ms의 RTT가 나타났습니다.

하지만 실제로 Servo가 움직이고 돌아가기까지 사용자는 약 1초를 기다리게 됩니다.

그래서 실제 Light ON / OFF 요청의 완료 시간을 별도로 측정했습니다.

20회 측정 결과 평균은 다음과 같았습니다.

구분	측정 시간
ON Mean	1032.498 ms
OFF Mean	1031.995 ms
Overall Mean	1032.247 ms

서보 제어 코드에는 다음 시간이 의도적으로 포함되어 있습니다.

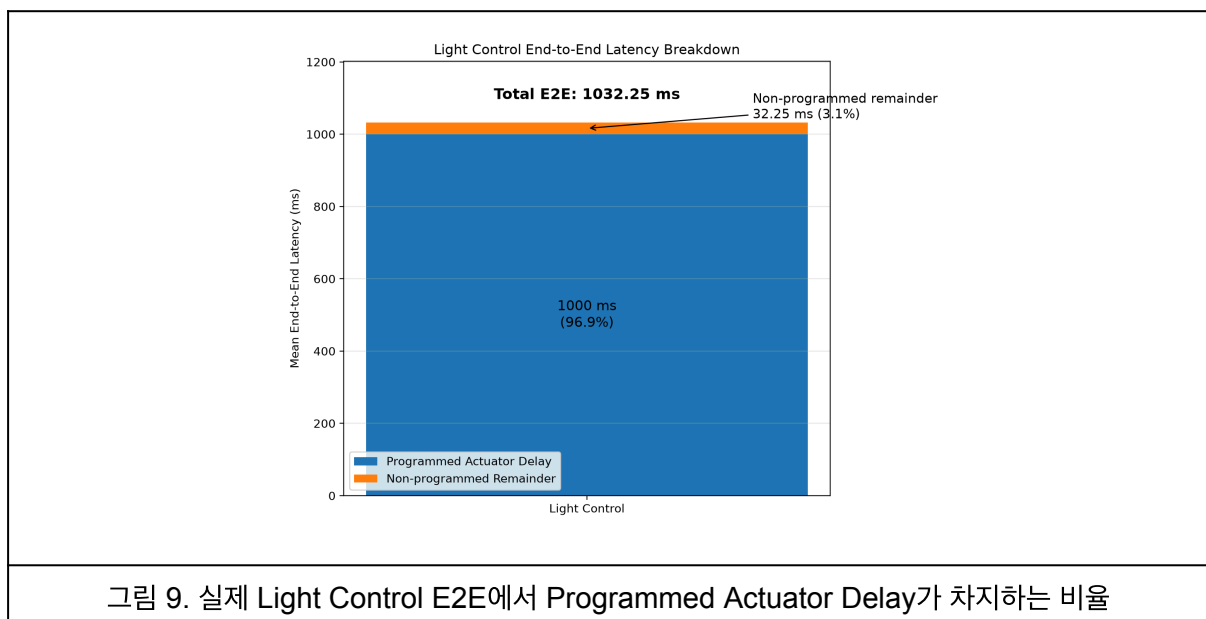
Press Hold : 400 ms

Return Wait : 600 ms

Total = 1000 ms

즉 전체 약 1032 ms 중 약 96.9%가 서보 모터를 안정적으로 눌렀다가 복귀시키기 위해 설정한 시간이었습니다.

따라서 현재 Light Switch의 사용자 체감 응답시간을 줄이기 위해서는 통신 지연을 몇 ms 줄이는 것보다, 먼저 안정성을 유지할 수 있는 범위에서 서보 모터의 Press Hold와 Return Wait를 줄이는 것이 더 큰 효과를 가질 것으로 판단하였습니다.



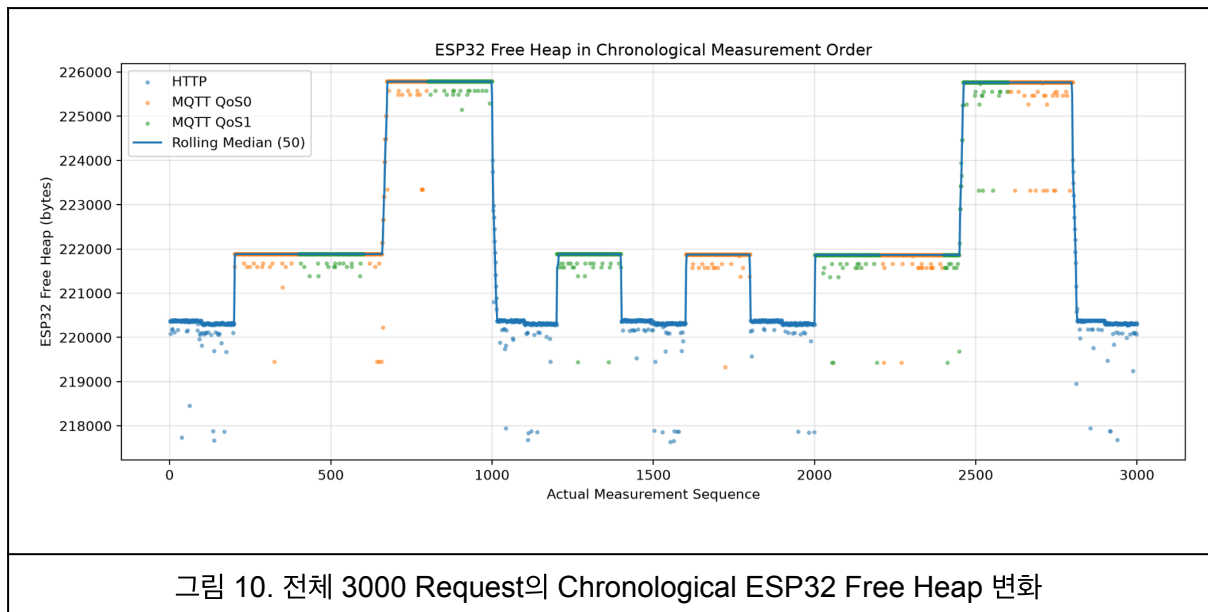
9. 추가로 확인한 내용

9.1 ESP32 Heap

Protocol별로 기록된 데이터를 처음 분석할 때는 각 파일의 첫 값과 마지막 값을 비교했습니다.

그런데 실제 실험에서는 Protocol 실행 순서를 Run마다 바꾸었기 때문에 이 방식은 전체 시간 흐름을 제대로 반영하지 못했습니다.

그래서 HTTP, MQTT QoS 0, MQTT QoS 1의 총 3000개 Sample을 실제 Timestamp 순서로 다시 정렬해 확인했습니다.



그 결과 Free Heap은 여러 수준 사이에서 변했지만, 실험이 진행될수록 계속 감소하는 형태는 관찰되지 않았습니다.

이 과정에서는 결과 자체보다 데이터를 어떤 순서로 해석하는지도 실험의 일부라는 점을 배웠습니다.

9.2 Raspberry Pi 자원 사용량

Raspberry Pi에서도 HTTP와 MQTT 실행 시 Process의 자원 사용량을 측정하였으며 다음과 같은 결과가 나타났습니다.

Protocol	Worker CPU	Worker RSS	Mosquitto CPU	Mosquitto RSS
HTTP	0.429%	23.165 MiB	0.008%	8.406 MiB
MQTT QoS 0	0.397%	23.371 MiB	0.037%	8.406 MiB
MQTT QoS 1	0.403%	23.389 MiB	0.043%	8.406 MiB

현재 수준의 요청에서는 Raspberry Pi 자원 사용량이 크지 않았습니다.

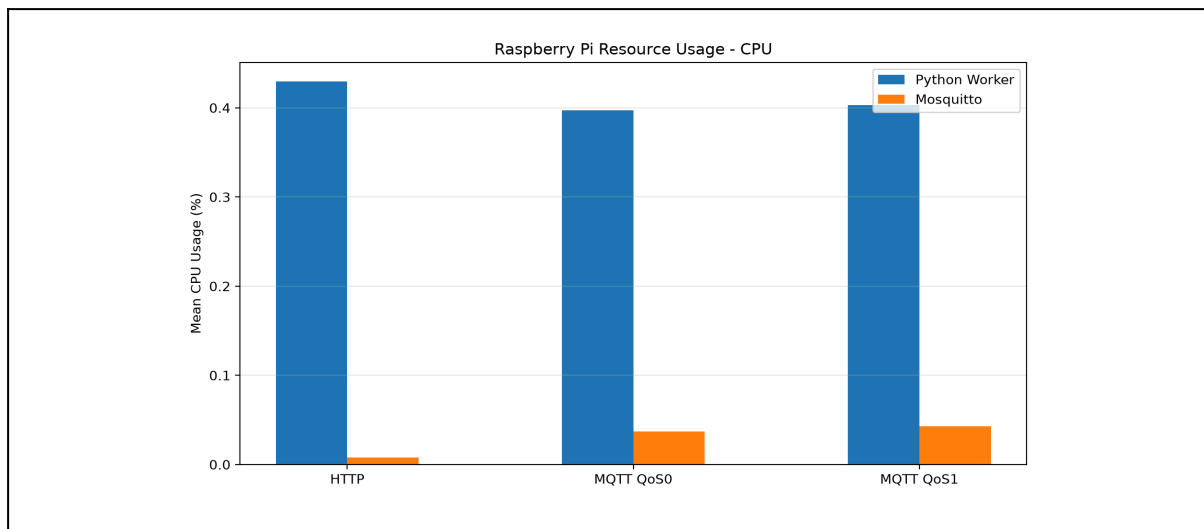


그림 11. Protocol별 Raspberry Pi Python Worker와 Mosquitto CPU Usage

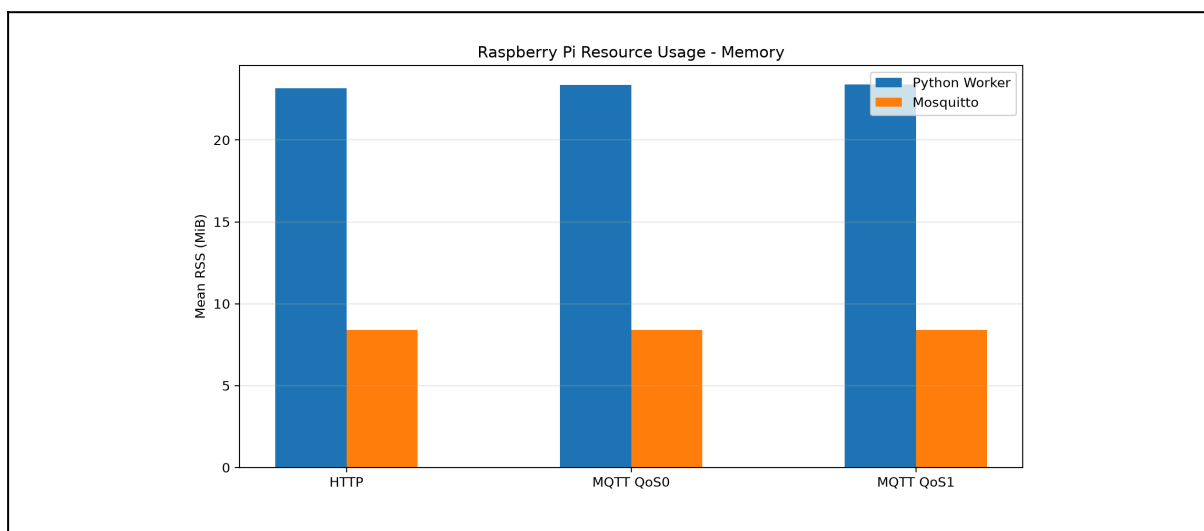


그림 12. Protocol별 Raspberry Pi Python Worker와 Mosquitto RSS Usage

현재 시스템을 동작시키는 조건에서 자원 사용량이 어느 정도인지 확인하기 위해 측정하였습니다.

10. 마무리

처음에는 기능을 추가하고, 의도한 대로 장치가 실제로 움직인다는 점 자체가 재미있었습니다. 전등이 켜지고, PC 전원 버튼이 눌리고, 웹에서 직접 제어할 수 있게 되는 것이 프로젝트의 가장 큰 목표라고 생각했습니다.

하지만 스위치가 제대로 눌리지 않고, MQTT의 RTT가 예상보다 크게 나오고, 처음 사용했던 분석 방법을 다시 수정하는 일을 겪으면서 기능 자체보다 “왜 이런 결과가 나타날까?”, “어디에서 시간이 사용되고 있을까?”가 더 궁금해졌습니다.

이번 프로젝트에서 궁금했던 모든 원인들을 해결한 것은 아닙니다. 오히려 설명하지 못한 결과를 만나면서 운영체제, 시스템, 네트워크, 컴퓨터 구조를 더 공부해야겠다는 생각이 들었습니다.

Raspberry Pi와 ESP32를 이용해 실제 생활 장치를 제어해보고 싶다는 생각 하나로 시작했지만, 프로젝트를 진행하면서 구현 → 문제 발견 → 원인 추정 → 측정 → 결과 확인 → 조건 수정 → 재실험 → 해석의 과정을 반복하게 되었습니다. 그 과정에서 처음 만든 시스템을 이전보다 훨씬 깊게 이해할 수 있었고, 앞으로는 시스템 내부에서 시간이 어디에 쓰이고 자원과 하드웨어 구조가 성능에 어떤 영향을 주는지를 더 깊게 공부하고 싶다는 목표도 생겼습니다.

단순히 동작하는 시스템을 만든 것을 넘어, 왜 성공하고 왜 실패하는지 직접 측정하고 확인해본 경험이라는 점에서 이번 프로젝트가 가장 의미 있게 남았습니다.